C

Coach Drill Designer
Full Technical Documentation

Complete architecture, feature set, data models, editor system & improvement roadmap

Next.js 14.2 App Router | TypeScript | React-Konva Canvas | Zustand State Management | @dnd-kit Drag & Drop | Tailwind CSS | localStorage Persistence | Print System

PLATFORM	VERSION	DATE	STACK
CoachDesigner	Phase 10 (MVP)	March 2026	Next.js 14 + Konva + Zustand

Table of Contents

1. Executive Summary

Coach Drill Designer is a fully client-side, browser-based football coaching platform built with Next.js 14 (App Router), TypeScript, and React. It enables coaches to visually design training drills on a canvas, build structured training sessions, plan seasonal schedules, and manage teams — all without a backend server or database.

All state is managed through Zustand stores with localStorage persistence, making the application fully offline-capable. The canvas editor is powered by **react-konva** (Konva.js), a high-performance 2D canvas engine, loaded exclusively on the client side to avoid Next.js SSR conflicts.

Platform Type

Single-Page Application (SPA) with file-based routing via Next.js App Router. No backend. No database. All data lives in localStorage.

Core Value

Visual drill design with a canvas editor, session planning with drag-and-drop timeline, and a full seasonal planning system — all in one tool.

Development Status

Phase 10 MVP. Build passes cleanly. Zero TypeScript errors. All features production-ready for local/offline use.

Key Statistics

Metric	Value
Framework	Next.js 14.2.5 (App Router)
Language	TypeScript 5.5
Canvas Engine	Konva 9.3 via react-konva 18.2
State Management	Zustand 4.5 with persist middleware
Drag & Drop	@dnd-kit/core 6.1 + @dnd-kit/sortable 7.0
Styling	Tailwind CSS 3.4 with custom brand tokens
localStorage Keys	6 keys (drills-v2, sessions-v2, teams-v1, folders-v1, season-plans-v1, calendar-v1)
Canvas Object Types	11 (player, cone, ball, goal, arrow, zone, circle, rectangle, line, curved, link)

Metric	Value
Pages / Routes	15 routes across 6 modules
Undo Stack Depth	30 levels per step context
Default Squad Size	25 outfield + 3 GK per side

2. System Architecture

2.1 Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js App Router	14.2.5	File-based routing, React Server Components shell, client components for all interactive UI
Language	TypeScript	5.5.4	Full type safety across all components, stores, and models
Canvas	react-konva / konva	18.2 / 9.3.14	2D canvas rendering for drill diagrams, loaded via `dynamic()` with `ssr:false`
State	Zustand + persist	4.5.4	6 isolated stores, each with auto-persist to localStorage via middleware
Drag & Drop	@dnd-kit	6.1 / 7.0	Accessible drag-and-drop for session timeline reordering and drill library sorting
Styling	Tailwind CSS	3.4.6	Utility-first with brand color tokens (`brand-orange` , `brand-dark`)
Config	next.config.mjs	—	.mjs format required (Next 14.2 does not support .ts config)

2.2 Folder Structure

```
sessionbuilder/
├── src/
│   ├── app/                        # Next.js App Router
│   │   ├── layout.tsx              # Root layout — NavBar + page slot
│   │   ├── page.tsx                # Redirects → /drills
│   │   ├── drills/
│   │   │   ├── page.tsx            # Drill library list
│   │   │   └── [id]/
│   │   │       ├── page.tsx        # Drill editor
│   │   │       └── view/page.tsx   # Fullscreen drill view
│   │   ├── sessions/
│   │   │   ├── page.tsx            # Sessions list
│   │   │   └── [id]/
│   │   │       ├── page.tsx        # Session builder
│   │   │       ├── view/page.tsx   # Session field view
│   │   │       └── print/page.tsx  # Session print preview
```

```
│   │   ├── teams/
│   │   │   ├── page.tsx              # Teams list
│   │   │   └── [id]/page.tsx         # Team editor
│   │   ├── season-plans/
│   │   │   ├── page.tsx              # Season plans list
│   │   │   └── [id]/
│   │   │       ├── page.tsx          # Season plan editor
│   │   │       └── view/page.tsx     # Season plan view/print
│   │   └── calendar/
│   │       └── page.tsx              # Calendar month view
│   │
│   ├── components/
│   │   ├── drill-editor/             # Editor engine (7 components)
│   │   │   ├── DrillEditorPage.tsx   # Main orchestrator
│   │   │   ├── PitchCanvas.tsx       # Konva canvas (SSR-disabled)
│   │   │   ├── PaletteSidebar.tsx    # Left tool palette
│   │   │   ├── InspectorPanel.tsx    # Right property inspector
│   │   │   ├── DrillEditorTopBar.tsx # Top toolbar
│   │   │   ├── DrillMetaPanel.tsx    # Drill info drawer
│   │   │   ├── PlayerDock.tsx        # Bottom player panel
│   │   │   └── FormationPicker.tsx   # Formation modal
│   │   ├── session-builder/          # Session builder (3 components)
│   │   │   ├── SessionBuilderPage.tsx
│   │   │   ├── SessionTimeline.tsx
│   │   │   └── DrillPicker.tsx
│   │   ├── season-plans/             # Season planning (3 components)
│   │   │   ├── SeasonPlanEditor.tsx
│   │   │   ├── SeasonPlansList.tsx
│   │   │   └── SeasonPlanPrintView.tsx
│   │   ├── calendar/
│   │   │   └── CalendarPage.tsx
│   │   ├── views/                    # Read-only view modes
│   │   │   ├── DrillView.tsx         # Fullscreen drill with keyboard nav
│   │   │   ├── SessionView.tsx       # Session field view
│   │   │   └── SessionPrintView.tsx  # Print-ready session layout
│   │   ├── team/
│   │   │   └── TeamPage.tsx
│   │   ├── DrillsList.tsx            # Drill library with folders/D&D
│   │   ├── SessionsList.tsx
│   │   ├── TeamsList.tsx
│   │   ├── MiniPitchPreview.tsx      # SVG thumbnail of drill canvas
│   │   └── NavBar.tsx                # Left sidebar navigation
│   │
│   ├── store/                        # Zustand stores
│   │   ├── drillsStore.ts
│   │   ├── sessionsStore.ts
│   │   ├── teamsStore.ts
│   │   ├── foldersStore.ts
│   │   ├── seasonPlansStore.ts
│   │   └── calendarStore.ts
│   │
│   ├── types/
│   │   └── index.ts                  # All TypeScript types (single source)
│   └── lib/
│       ├── seed.ts                   # Seed data for first load
│       └── sessionQuality.ts         # Session analysis utility
```

```
│
├── next.config.mjs
├── tailwind.config.ts
└── package.json
```

2.3 Module Responsibilities

Module	Files	Responsibility
Editor Engine	`drill-editor/`	All drill canvas editing: object creation, selection, snapping, drawing tools, undo/redo, step management, formation placement
Session Builder	`session-builder/`	Building training sessions from drills: timeline with section groups, intensity setting, drag reorder, session summary
Season Planner	`season-plans/`	Long-term planning: assign sessions to calendar dates, generate printable weekly plan view
Calendar	`calendar/`	Month-view calendar of all events (training, match, rest), synced with season plan entries
Teams	`team/`	Team management: colors, player roster, training days; colors auto-inherit to drill canvas players
Print System	`views/`	Brand-styled A4 landscape print views for sessions (one drill per page) and season plans (week-by-week table)
State Layer	`store/`	6 isolated Zustand stores, each with persist middleware writing to separate localStorage keys
Types	`types/index.ts`	Single source of truth for all TypeScript interfaces and union types

2.4 State Management Architecture

> Every store follows the same pattern: **Zustand `create()` wrapped in `persist()`** middleware with a custom SSR-safe storage adapter that guards all `localStorage` access behind a `typeof window !== 'undefined'` check.

Store	localStorage Key	State Shape	Key Actions
`drillsStore`	`coach-drills-v2`	`Record<string, Drill>`	addDrill, updateDrill, deleteDrill, duplicateDrill, addObject, updateObject, deleteObject, setObjects, addDrillStep, setStepObjects, removeDrillStep, toggleFavorite

Store	localStorage Key	State Shape	Key Actions
`sessionsStore`	`coach-sessions-v2`	`Record<string, Session>`	addSession, updateSession, deleteSession, duplicateSession, addBlock, updateBlock, deleteBlock, reorderBlocks
`teamsStore`	`coach-teams-v1`	`Record<string, Team>`	addTeam, updateTeam, deleteTeam, addPlayer, updatePlayer, deletePlayer
`foldersStore`	`coach-folders-v1`	`Record<string, DrillFolder>` + subcategories	addFolder, updateFolder, deleteFolder (cascades subs), addSubcategory, updateSubcategory, deleteSubcategory
`seasonPlansStore`	`coach-season-plans-v1`	`Record<string, SeasonPlan>`	addPlan, updatePlan, deletePlan, upsertEntry, deleteEntry
`calendarStore`	`coach-calendar-v1`	`Record<string, CalendarEvent>`	addEvent, updateEvent, deleteEvent, bulkAddEvents, syncSeasonPlanEntry, clearSeasonPlanEntry

2.5 Persistence & Seeding

Auto-Persistence

Zustand `persist` middleware automatically serializes the entire store state to `localStorage` as JSON on every state mutation. Rehydration happens automatically on first render.

The SSR-safe adapter wraps all access so Next.js server-side rendering never crashes attempting to access `window.localStorage`.

Seed Data

`src/lib/seed.ts` provides `buildSeedDrills()`, `buildSeedSessions()`, and `buildSeedTeams()` functions. These are called once via `seedIfEmpty()` on each page mount — only if the store is empty and `_seeded` flag is false. This ensures first-time users see populated examples.

3. Architecture Diagrams

3.1 System Architecture Diagram

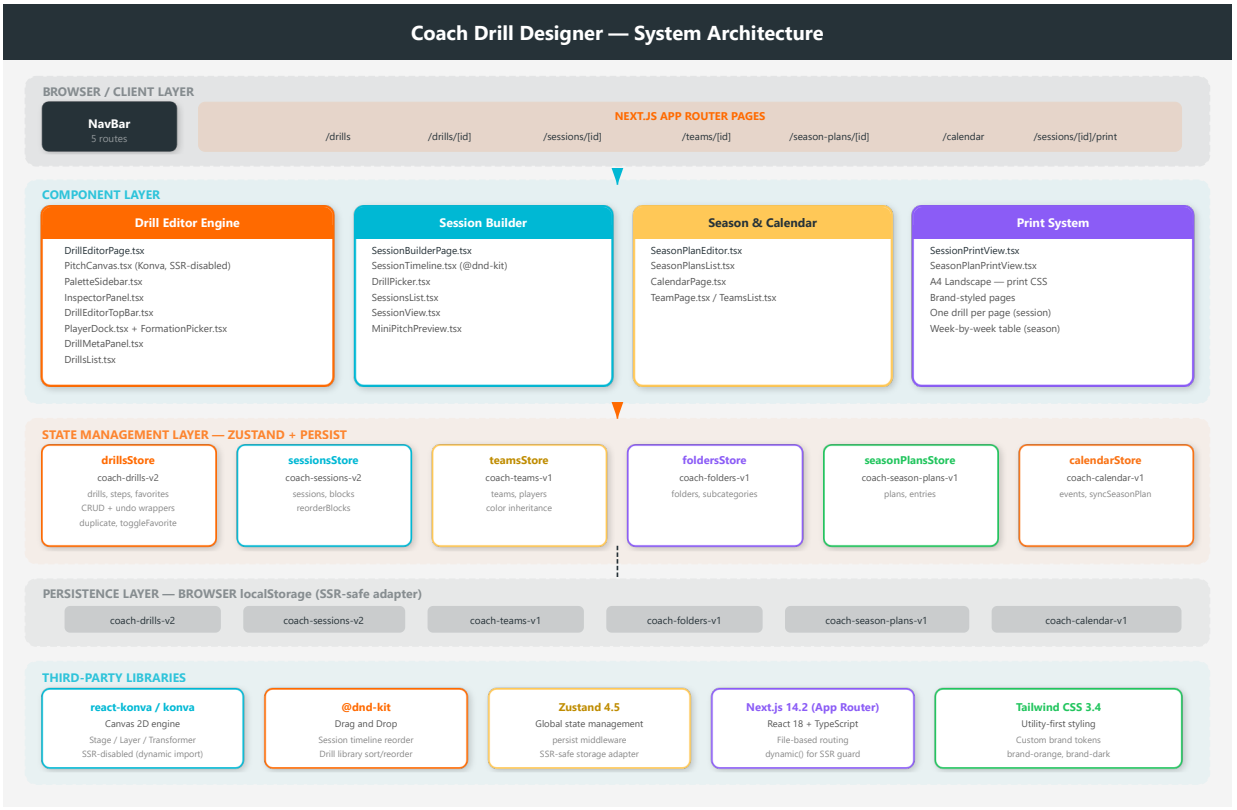

Figure 1 — Full system architecture showing browser layer, component layer, state management, persistence, and third-party libraries

The diagram illustrates the layered architecture: the **Browser/Client Layer** hosts all Next.js pages routed by the App Router. Beneath that, the **Component Layer** is divided into four functional groups — the Drill Editor Engine, Session Builder, Season & Calendar, and Print System. All components share state through the **Zustand State Management Layer** which automatically persists to the **localStorage Persistence Layer** using six isolated keys.

3.2 Editor Engine Diagram

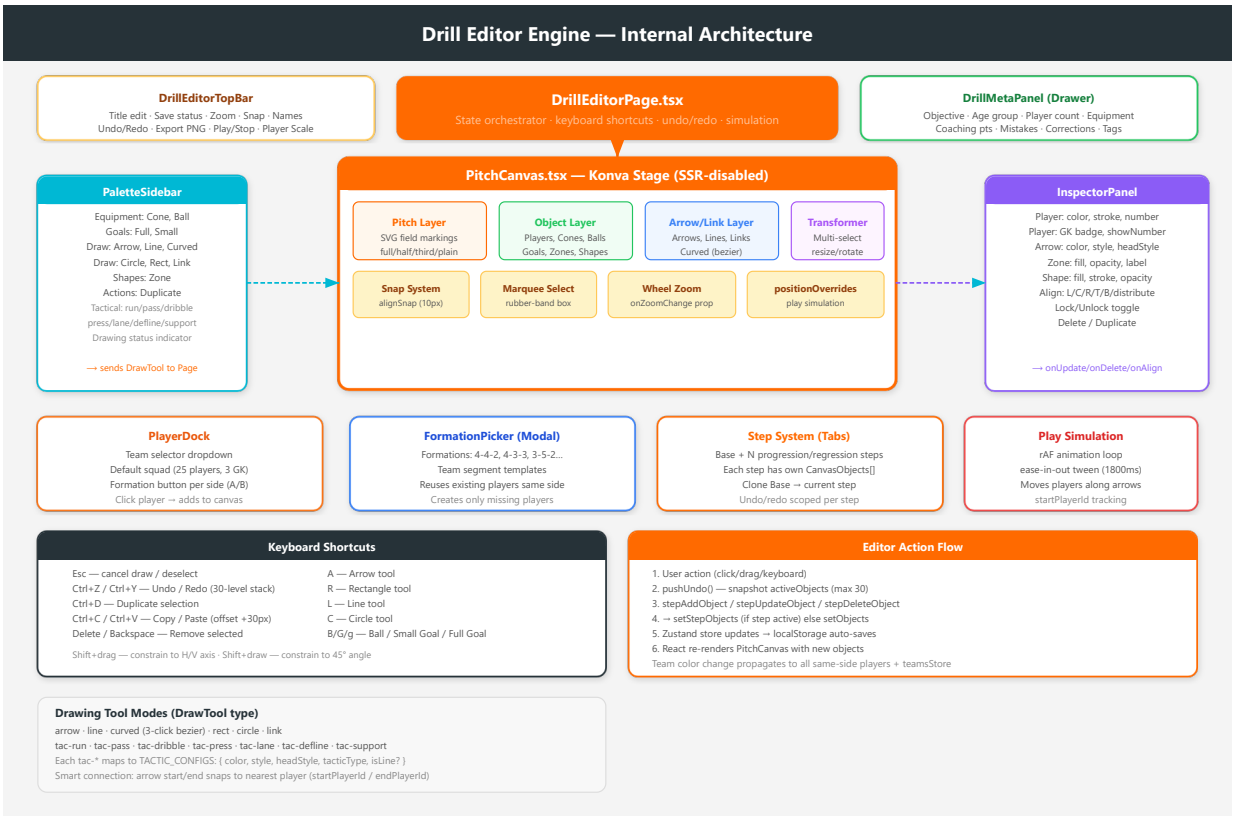

Figure 2 — Internal structure of the Drill Editor Engine showing component relationships, canvas layers, drawing tools, and data flow

`DrillEditorPage.tsx` is the central orchestrator. It owns all editor state (selected objects, draw tool, undo stack, zoom, step context) and passes handlers down as props. The **PitchCanvas** is the core rendering engine: it splits rendering into a pitch layer, object layer, arrow/link layer, and transformer layer. Surrounding components (PaletteSidebar, InspectorPanel, PlayerDock, FormationPicker) communicate only through callbacks — they never write to the store directly.

3.3 Data Flow Diagram

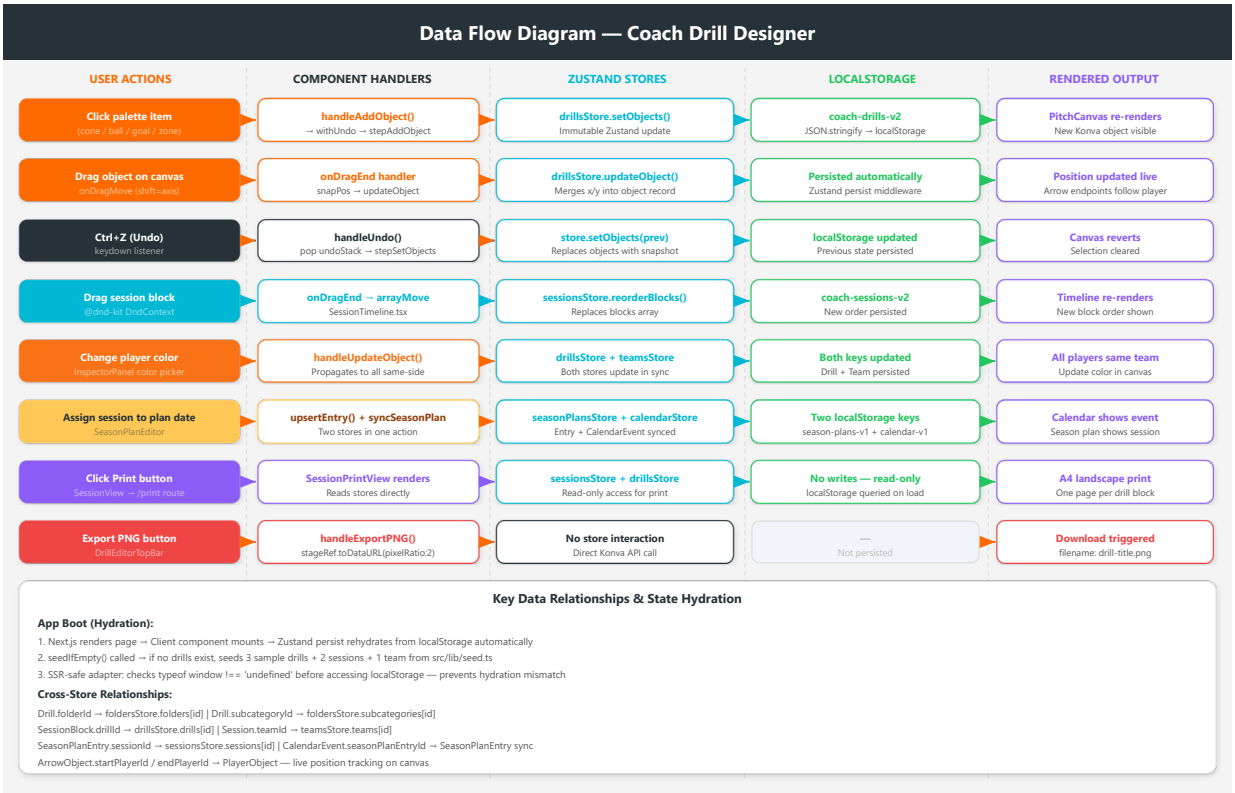

Figure 3 — Data flow for all major user actions: add object, drag/move, undo, session reorder, team color change, season plan, print, and PNG export

Every mutating user action follows the same five-step flow: *User Action → Component Handler → Zustand Store mutation → localStorage auto-persist → React re-render*. Read-only operations (Print, Export PNG) bypass the store write path entirely. Cross-store operations (e.g. assigning a session to a season plan date) atomically update two stores in one handler.

4. Data Models

All types are defined in a single source-of-truth file: `src/types/index.ts`. Every store and component imports exclusively from this file.

4.1 Drill

> The central entity. A drill is a canvas diagram with rich coaching metadata.

```
interface Drill {
  id: string;
  title: string;
  description?: string;
  pitch: Pitch;                    // { type, width, height, colors? }
  objects: CanvasObject[];         // Base layer objects

  // Coaching metadata
  objective?: string;
  ageGroup?: string;
  playerCount?: string;
  areaSize?: string;
  durationMin?: number;
  equipment?: string[];
  coachingPoints?: string[];
  coachingCues?: string[];
  commonMistakes?: string[];
  corrections?: string[];
  keyConstraints?: string[];
  progression?: string;
  regression?: string;
  notes?: string;

  // Organisation
  folderId?: string | null;
  subcategoryId?: string | null;
  sortOrder?: number;              // Manual drag-to-reorder position
  tags?: string[];
  teamId?: string | null;
  trainingDay?: string | null;

  // Visual
  playerScale?: number;           // 0.5–2.0, global player size multiplier

  // Relationships
```

```
  parentDrillId?: string | null;
  relationType?: 'base'|'variation'|'progression'|'regression' | null;
  isFavorite?: boolean;

  // Multi-step system
  steps?: DrillStep[];            // Progression/regression steps

  createdAt: string;              // ISO timestamp
  updatedAt: string;
}
```

4.2 DrillStep

```
interface DrillStep {
  id: string;
  label: string;                  // e.g. "Progression", "Regression", "Step 3"
  objects: CanvasObject[];        // Independent object set for this step
}
```

Step System: A drill can have a Base layer (Drill.objects) and N additional DrillStep objects. Each step has its own complete independent CanvasObject array, allowing coaches to show progressions without creating separate drills. Steps are labelled automatically (Progression, Regression, Step N). Switching steps clears the undo stack.

4.3 Pitch

```
type PitchType = 'full' | 'half' | 'third' | 'plain';

interface Pitch {
  type: PitchType;
  width: number;     // Logical pixels: full=840, half=840, third=840
  height: number;    // Logical pixels: full=540, half=420, third=300
  colors?: {
    grass: string;
    grassSecondary?: string;   // Alternate stripe color
    lines: string;
  };
}
```

4.4 CanvasObject Union

The `CanvasObject` type is a discriminated union of 11 object types, distinguished by the `type` field:

```
type CanvasObject =
  | PlayerObject | ConeObject | BallObject | GoalObject
```

```
  | ArrowObject | ZoneObject | CircleShapeObject | RectangleObject
  | LineObject | CurvedLineObject | LinkObject;
```

Type	Key Fields	Description
`'player'`	x, y, color, number, name, team ('A'|'B'), showNumber, strokeColor, numberColor, teamColorInherited, rotation, locked	Jersey-style player dot. GK detected by number 1/12/13 → gold fill. teamColorInherited enables auto-sync with team store.
`'cone'`	x, y, color, rotation, locked	Training cone placed as a colored triangle/polygon.
`'ball'`	x, y, locked	Football (soccer ball), white with black pattern.
`'goal'`	x, y, size ('small'|'full'), rotation, locked	Full-size or small/mini goal rendered as a rectangle.
`'arrow'`	startX, startY, endX, endY, color, style ('solid'|'dashed'), headStyle ('filled'|'open'), strokeWidth, tacticType?, startPlayerId?, endPlayerId?, locked	2-point arrow. startPlayerId/endPlayerId enable live tracking: arrow endpoints follow player positions. tacticType assigns football-specific visual style.
`'line'`	startX, startY, endX, endY, color, strokeWidth, dashed, tacticType?, locked	Straight line without arrowhead. Used for tactical lines (defensive line, lane).
`'curved'`	startX, startY, cpX, cpY, endX, endY, color, strokeWidth, dashed, locked	Quadratic bezier curve. 3-click creation: click start → click end → click control point.
`'zone'`	x, y, width, height, fill, opacity (0–1), label, strokeColor, strokeWidth, rotation, locked	Filled rectangular area, used to mark pitch zones or areas of play.
`'circle'`	x, y, radius, stroke, strokeWidth, fill, fillOpacity, opacity, dashed, rotation, locked	Circle shape. Center + edge click creation.
`'rectangle'`	x, y, width, height, stroke, strokeWidth, fill, fillOpacity, opacity, dashed, rotation, locked	Rectangle shape. Corner-to-corner click creation.
`'link'`	fromPlayerId, toPlayerId, color, dashed, label, locked	Dynamic line connecting two players by ID. Renders as a line between live player positions.

4.5 TacticType

```
type TacticType = 'run' | 'pass' | 'dribble' | 'press' | 'lane' | 'defline' | 'support';
```

Tactic	Color	Style	Head	Object Type
run	#fbbf24 (amber)	solid	filled	arrow
pass	#ffffff	dashed	open	arrow
dribble	#f97316 (orange)	solid	filled	arrow
press	#ef4444 (red)	dashed	filled	arrow
support	#22c55e (green)	dashed	open	arrow
lane	#3b82f6 (blue)	dashed	open	line
defline	#8b5cf6 (purple)	solid	open	line

4.6 Session & SessionBlock

```
type Intensity = 'low' | 'mid' | 'high';
type SessionSection = 'warmup' | 'main' | 'game' | 'cooldown';

interface SessionBlock {
  id: string;
  drillId: string;          // References Drill.id
  durationMin: number;
  intensity: Intensity;
  notes?: string;
  section?: SessionSection; // Which phase of the session
}

interface Session {
  id: string;
  title: string;
  date?: string;
  blocks: SessionBlock[];   // Ordered array of drill blocks
  objective?: string;
  ageGroup?: string;
  playerCount?: string;
  notes?: string;
  teamId?: string | null;
  trainingDay?: string | null;
  createdAt: string;
  updatedAt: string;
}
```

4.7 Team & TeamPlayer

```
interface TeamPlayer {
  id: string;
  name: string;
  number?: string;
  position?: string;
  color?: string;
  teamSide: 'home' | 'opponent';
}

interface Team {
  id: string;
  name: string;
  ageGroup: string;
  primaryColor: string;             // Home jersey color
  secondaryColor: string;
  opponentPrimaryColor: string;     // Away jersey color
  opponentSecondaryColor?: string;
  primaryStrokeColor?: string;      // Jersey ring color — home
  opponentStrokeColor?: string;     // Jersey ring color — away
  primaryNumberColor?: string;      // Number text color — home
  opponentNumberColor?: string;     // Number text color — away
  trainingDays: string[];           // ["Sunday","Tuesday","Thursday"]
  players: TeamPlayer[];
  createdAt: string;
  updatedAt: string;
}
```

4.8 DrillFolder & FolderSubcategory

```
interface DrillFolder {
  id: string; name: string; createdAt: string; updatedAt: string;
}

interface FolderSubcategory {
  id: string;
  folderId: string;         // Parent folder reference
  name: string;
  createdAt: string;
  updatedAt: string;
}
```

Deleting a folder cascades to delete all its subcategories via `foldersStore.deleteFolder()`. Drills that were in the folder keep their `folderId` value but the folder is gone — they become "unfoldered".

4.9 SeasonPlan & SeasonPlanEntry

```
interface SeasonPlanEntry {
  id: string;
  date: string;             // ISO date string
  trainingDay?: string;
  sessionId?: string;       // Linked session
  notes?: string;
}

interface SeasonPlan {
  id: string;
  title: string;
  teamId: string;           // Linked team
  startDate: string;
  endDate: string;
  entries: SeasonPlanEntry[];
  createdAt: string;
  updatedAt: string;
}
```

4.10 CalendarEvent

```
type CalendarEventType = 'training' | 'match' | 'rest' | 'other';
type CalendarEventStatus = 'planned' | 'completed' | 'cancelled';

interface CalendarEvent {
  id: string;
  title: string;
  date: string;
  type: CalendarEventType;
  teamId?: string;
  sessionId?: string;
  notes?: string;
  status?: CalendarEventStatus;
  seasonPlanEntryId?: string;  // Links to SeasonPlanEntry for sync
}
```

When a session is assigned to a SeasonPlanEntry, `calendarStore.syncSeasonPlanEntry()` creates or updates a corresponding CalendarEvent automatically, keeping the calendar in sync with the season plan.

5. Page Structure & Routing

Route	Component	Purpose
`/`	page.tsx	Instant redirect to `/drills`
`/drills`	DrillsList.tsx	Full drill library with folders, subcategories, favorites, search, drag-to-reorder, 2-step new drill modal with 8 templates
`/drills/[id]`	DrillEditorPage.tsx	Full canvas editor with all drawing tools, inspector, player dock, step tabs, simulation
`/drills/[id]/view`	DrillView.tsx	Fullscreen presentation mode; keyboard shortcuts: F=fullscreen, ←→=prev/next drill navigation
`/sessions`	SessionsList.tsx	Session cards with duration/intensity summary, mini previews, create/duplicate/delete
`/sessions/[id]`	SessionBuilderPage.tsx	Session builder with timeline, drill picker, section headers, session quality analysis
`/sessions/[id]/view`	SessionView.tsx	Read-only session view with drill progression, print button
`/sessions/[id]/print`	SessionPrintView.tsx	Brand-styled print layout: cover page + one A4 landscape page per drill block
`/teams`	TeamsList.tsx	Team cards, create new team
`/teams/[id]`	TeamPage.tsx	Team editor: colors, roster management, training days configuration
`/season-plans`	SeasonPlansList.tsx	Season plan cards, create new plan
`/season-plans/[id]`	SeasonPlanEditor.tsx	Weekly grid: assign sessions to dates, auto-syncs to calendar
`/season-plans/[id]/view`	SeasonPlanPrintView.tsx	Week-by-week table with drill thumbnails, brand styled, printable
`/calendar`	CalendarPage.tsx	Month-view calendar with event types (training/match/rest/other), status tracking

6. Drill Library System

6.1 Drill List View

The drill library (`DrillsList.tsx`) is the main hub for managing all drills. It provides:

Organisation

- Folders sidebar (left panel)
- Subcategories nested under folders
- Favorites section at top (starred drills)
- Unfoldered section at bottom
- Drill cards sortable within a folder/subcategory

Drill Cards

- MiniPitchPreview SVG thumbnail
- Title, description, duration
- Step count badge (if steps exist)
- Star toggle for favorites
- Edit / Duplicate / Delete actions

6.2 New Drill Modal — 8 Templates

A 2-step creation modal: first choose a template, then set the pitch type.

Template	Description
Blank	Empty canvas, user starts from scratch
Rondo	Possession circle exercise template
Possession	Keep-ball drill setup
Build-up	Playing out from the back
Finishing	Shooting / finishing exercise
Pressing	High press / gegenpressing setup
Transition	Attack/defense transition drill
SSG	Small-sided game setup

6.3 Folders & Subcategories

Folders are top-level organisational containers. Each folder can have **subcategories** — child groupings within the folder. Drills store both `folderId` and `subcategoryId` fields. Deleting a folder cascades to delete all its subcategories.

6.4 Drag-to-Reorder

Drill cards within a folder/subcategory are sortable using **@dnd-kit/sortable**. Each card uses the `useSortable` hook. Drag end triggers a `sortOrder` update in the drills store. Drill cards can also be dragged between folders/subcategories.

6.5 Favorites

Any drill can be starred via the star icon on the card or the `toggleFavorite` store action. Starred drills appear in a dedicated **Favorites** section at the top of the library regardless of their folder assignment.

7. Editor Engine

The editor engine is implemented across the `src/components/drill-editor/` directory. `DrillEditorPage.tsx` is the orchestrator that owns all state. `PitchCanvas.tsx` is the Konva canvas renderer, loaded via `dynamic()` with `ssr: false` to prevent server-side rendering conflicts.

7.1 Canvas Architecture (Konva)

> `PitchCanvas.tsx` renders a **Konva Stage** containing multiple **Layers** in a defined z-order. All objects are rendered as Konva nodes (Rect, Circle, Arrow, Line, Path, Text, Group).

Layer (top → bottom)	Contents
Transformer Layer	Konva Transformer node for resize/rotate handles on selected objects
Link/Arrow Layer	ArrowObject, LineObject, CurvedLineObject (rendered below players so they appear "under" player tokens)
Object Layer	PlayerObject, ConeObject, BallObject, GoalObject, ZoneObject, CircleShapeObject, RectangleObject
Pitch Layer	Field markings (grass rectangles, white lines, penalty areas, circles, arcs) rendered from pitch.type

The **Stage** handles mouse events for drawing tools (onMouseDown, onMouseMove, onMouseUp) and zoom (onWheel). Drag events on individual nodes are handled per-object for position updates.

7.2 Drawing Tools

The active drawing tool is stored as `DrawTool` in `DrillEditorPage` state. All tool buttons in `PaletteSidebar` and all keyboard shortcuts toggle this value.

Tool	Clicks Required	Object Created	Keyboard
arrow	2 (start → end)	ArrowObject (red, solid, filled)	A
line	2 (start → end)	LineObject (white)	L
curved	3 (start → end → control point)	CurvedLineObject (bezier)	—

Tool	Clicks Required	Object Created	Keyboard
rect	2 (corner → opposite corner)	RectangleObject	R
circle	2 (center → edge)	CircleShapeObject	C
link	2 (player A → player B)	LinkObject	—
tac-run	2	ArrowObject (amber, solid)	—
tac-pass	2	ArrowObject (white, dashed, open)	—
tac-dribble	2	ArrowObject (orange, solid)	—
tac-press	2	ArrowObject (red, dashed)	—
tac-support	2	ArrowObject (green, dashed, open)	—
tac-lane	2	LineObject (blue, dashed)	—
tac-defline	2	LineObject (purple, solid)	—

Curved Line — 3-Click Flow: Click 1 sets `drawFirstPoint` (start). Click 2 sets `drawSecondPoint` (end). Click 3's position becomes the bezier control point (`cpX`, `cpY`). The `CurvedLineObject` is then created and added to the store.

Shift Constraint: While any draw tool is active, holding Shift constrains the drawn line/arrow to 45° angle increments. While dragging a placed object, holding Shift constrains movement to the horizontal or vertical axis.

7.3 Smart Arrow Connections

When drawing an arrow or tactical arrow tool and the first click lands on a **PlayerObject**, the player's ID is stored as `arrowStartPlayerId`. If the second click also lands on a player, that player's ID is stored as `endPlayerId`. The resulting ArrowObject has `startPlayerId` and/or `endPlayerId` set. PitchCanvas reads these IDs on every render and substitutes the live player position for the arrow endpoints — so the arrow follows the player when they are dragged.

7.4 Selection System

Single Select

Clicking any object calls `handleSelect(id)`: sets `selectedId` and `selectedIds = [id]`. Clicking empty canvas deselects.

Multi-Select

Shift+click on any object calls `handleMultiSelect(id)` using a ref (`selectedIdsRef`) to avoid stale closure: toggles the object in

`selectedIds`. The last selected becomes `selectedId` (shown in inspector).

Marquee (Rubber-Band) Select

Drag on empty canvas to draw a selection rectangle. All objects whose centers fall within the rectangle are collected and passed to `handleMarqueeSelect(ids, additive)`. Additive (Shift held) merges with existing selection.

7.5 Snapping & Alignment

Alignment snap is always active when `snapToGrid` is true. During `onDragMove`, the system scans all other objects and computes candidate snap targets: their x/y positions, midpoints, and edges. If the dragged object's position is within **10 pitch pixels** of any snap target, it snaps to that coordinate. Visual guide lines are drawn on the canvas during drag.

The **Inspector Align toolbar** provides manual alignment: Left, Center-X, Right, Top, Center-Y, Bottom, Distribute-H, Distribute-V. These operate on all `selectedIds` (minimum 2 objects).

7.6 Undo / Redo System

The undo system is a client-side snapshot stack. Before any mutating operation, `pushUndo()` takes a copy of the current `activeObjects` array and pushes it onto `undoStack` (capped at 30 entries, oldest discarded). The redo stack is cleared on any new action. Both stacks are scoped to the active step — switching steps clears both stacks.

```
// Undo: pop previous state, push current to redo stack
handleUndo() → undoStack.pop() → stepSetObjects(prev) → redoStack.push(current)

// Redo: pop from redo stack, push current to undo stack
handleRedo() → redoStack.pop() → stepSetObjects(next) → undoStack.push(current)
```

7.7 Copy / Paste / Duplicate

Action	Shortcut	Behaviour
Copy	Ctrl+C	Copies selected objects (all selectedIds or selectedId) to `clipboardItems` state — no store write
Paste	Ctrl+V	Creates new objects from clipboard with new UUIDs, offset +30px on x/y. Arrow player connections are stripped (independent copy). Selects new objects.

Action	Shortcut	Behaviour
Duplicate	Ctrl+D or palette button	Same as copy+paste but immediate, offset +10px. Works on multi-select.

7.8 Step System (Tabs)

The step tab bar lives between the toolbar and the canvas. The **Base** tab always shows `drill.objects`. Each additional step shows its own `DrillStep.objects`. Adding a step auto-clones all Base objects into the new step. **Clone base** button (only in step mode) overwrites the current step's objects with fresh copies from Base.

All store operations are routed through **step-aware wrappers**: `stepAddObject`, `stepUpdateObject`, `stepDeleteObject`, `stepSetObjects`. These check `activeStepId` and call either `setStepObjects(drillId, stepId, objects)` or the direct store action based on context.

7.9 Play Simulation

The Play button becomes active when any ArrowObject with `startPlayerId` exists. On play:

1. Collects all arrows with `startPlayerId`, reads the current player position and arrow end position as movement targets.
2. Starts a **requestAnimationFrame loop** over 1800ms.
3. Applies ease-in-out tween: `t < 0.5 ? 2t² : -1+(4-2t)t`.
4. Updates `positionOverrides` state on every frame — a `Record<playerId, {x,y}>` map.
5. PitchCanvas substitutes overridden positions for those player nodes during rendering — players visually move.
6. On completion or Stop, overrides are cleared and players snap back to stored positions.

7.10 Player & Team System

Player Dock

Bottom panel showing all players for both teams (or default squad of 25+3 GK per side). Team selector dropdown at top. Clicking a player adds it to canvas center. Formation buttons per side open the Formation Picker.

Team Color Inheritance

Players with `teamColorInherited: true` auto-sync color when the linked team's color changes. Manually changing a player's color in the inspector sets `teamColorInherited: false` and also propagates to all same-side players + updates the team store.

GK Detection

> Players with number 1, 12, or 13 are automatically treated as goalkeepers. The Inspector Panel shows a GK badge and Konva renders them with a gold fill instead of their team color.

7.11 Formation Picker

A modal component with a formation list (4-4-2, 4-3-3, 3-5-2, and team segment templates like back-4, mid-3, front-3). Formations use **landscape percentage coordinates** [lateral_pct, depth_pct] that are scaled to the current pitch dimensions. When placing a formation, the system **reuses existing players** on the same side (repositions them), creating only the missing ones — preventing duplicate players from accumulating.

7.12 Inspector Panel

Object Type	Inspector Controls
Player	Fill color, stroke color, number color, number text, name, GK badge, show/hide number toggle, lock toggle, delete, duplicate
Arrow	Color, line style (solid/dashed), head style (filled/open), stroke width, lock toggle
Zone	Fill color, opacity slider (0–1), label text, stroke color, stroke width, lock toggle
Circle	Stroke color, stroke width, fill color, fill opacity, overall opacity slider, dashed toggle, lock toggle
Rectangle	Stroke color, stroke width, fill color, fill opacity, overall opacity slider, dashed toggle, lock toggle
Cone	Fill color, lock toggle
Line/Link	Color, stroke width, dashed toggle, lock toggle
Multi-select (2+)	Alignment toolbar: left, center-X, right, top, center-Y, bottom, distribute-H, distribute-V

7.13 Keyboard Shortcuts

Key	Action
Esc	Cancel active draw tool, clear first point, deselect
Ctrl+Z	Undo (30-level stack, scoped per step)
Ctrl+Y / Ctrl+Shift+Z	Redo
Ctrl+D	Duplicate selected objects (+10px offset)
Ctrl+C	Copy selected objects to clipboard

Key	Action
Ctrl+V	Paste clipboard (+30px offset)
Delete / Backspace	Delete selected objects (when not in text input)
A	Activate Arrow tool
R	Activate Rectangle tool
L	Activate Line tool
C	Activate Circle tool
B	Place Ball at canvas center (with ±20px jitter)
g (lowercase)	Place Small Goal at canvas center
G (uppercase)	Place Full Goal at canvas center
D (with arrow selected)	Toggle arrow style solid ↔ dashed
Shift+drag object	Constrain movement to horizontal or vertical axis
Shift+draw	Constrain drawn line to 45° angle increments
Mouse wheel	Zoom canvas in/out

7.14 DrillEditorTopBar Controls

Inline Title Edit | Save Status Indicator | Zoom Controls | Snap Toggle | Show Names Toggle | Undo / Redo Buttons | Player Scale Slider (0.5–2×) | Export PNG (2× pixel ratio) | Play / Stop Simulation | Drill Info Drawer

7.15 Drill Meta Panel

A slide-out drawer (right side) for non-canvas drill metadata:

- Objective, Age Group, Player Count, Area Size
- Duration (minutes)
- Equipment list
- Coaching Points (bullet list)
- Coaching Cues
- Common Mistakes
- Corrections
- Key Constraints
- Progression / Regression (text)
- Tags, Team link, Training Day, Notes

8. Session Builder

8.1 Timeline Structure

The session timeline (`SessionTimeline.tsx`) displays all blocks in a vertical chronological list. Blocks are grouped by `SessionSection`: Warm-up, Main, Game, Cool-down. Each section has a coloured header (sky, orange, amber, violet) with an icon and total duration.

> The timeline uses **@dnd-kit**: each `BlockCard` is wrapped in `useSortable()` from `@dnd-kit/sortable`. The outer list is wrapped in `DndContext` with `closestCenter` collision detection. Drag end fires `arrayMove()` and calls `reorderBlocks(sessionId, newBlocks)` on the sessions store.

8.2 Block Card

Each block card shows:

- Drag handle (left edge)
- MiniPitchPreview SVG thumbnail
- Drill title + folder name
- Duration (editable inline)

- Intensity badge (low/mid/high) with color
- Section phase selector dropdown
- Per-block notes field
- Duplicate / Delete actions

8.3 Intensity System

Intensity	Color	Badge Style
Low	sky-400 (#38bdf8)	sky-50 bg, sky-200 border
Mid	amber-400 (#fbbf24)	amber-50 bg, amber-200 border
High	red-400 (#f87171)	red-50 bg, red-200 border

8.4 Session Summary (Right Panel)

The session builder right panel shows a live summary:

- Total session duration (sum of all block durations)
- Breakdown by section (warmup/main/game/cooldown) — total minutes each

- Intensity distribution bar (count of low/mid/high blocks)
- Aggregated equipment list from all linked drills
- Session quality score via `src/lib/sessionQuality.ts`

8.5 Drill Picker

`DrillPicker.tsx` is a modal/panel that lists all available drills with search and folder filter. Selecting a drill creates a new `SessionBlock` with a default 15-minute duration and mid intensity, appended to the session's blocks array.

9. Season Planner

9.1 Overview

The Season Planner (`SeasonPlanEditor.tsx`) provides a long-term training calendar for a team. A plan spans a date range (start → end) and belongs to one team. It renders as a week-by-week grid where each training day cell can have a session assigned.

9.2 Entry Management

> Clicking a date cell opens a session picker. Selecting a session calls `seasonPlansStore.upsertEntry()` to save the assignment, and simultaneously calls `calendarStore.syncSeasonPlanEntry()` to create or update a `CalendarEvent` linked via `seasonPlanEntryId`. This two-store sync keeps the Calendar view automatically up to date.

9.3 Season Plan Print View

The `SeasonPlanPrintView.tsx` component renders a **week-by-week table**:

- Header row: week number + date range (Mon–Sun)
- Each training day in that week as a column cell
- Each cell shows: session title, intensity badge, a `MiniPitchPreview` thumbnail of the first drill in that session
- Blank cells for non-training days
- Brand colors throughout: orange headers, dark sidebar, accent accents
- Designed for A4 landscape print output

Session intensity is derived by `sessionIntensity(session)`: averages block intensities (low=1, mid=2, high=3) and rounds to nearest level.

10. Calendar

10.1 Month View

`CalendarPage.tsx` renders a full month grid calendar. Days with events show event chips coloured by type:

Event Type	Display
training	Brand orange chip
match	Red chip
rest	Gray chip
other	Accent chip

10.2 Event Status

Each event has a `CalendarEventStatus`: `'planned'`, `'completed'`, or `'cancelled'`. This can be changed directly in the calendar view. Cancelled events render with a strikethrough.

10.3 Season Plan Sync

Events created from season plan entries carry a `seasonPlanEntryId` field. The `calendarStore.syncSeasonPlanEntry()` method performs an **upsert**: it searches for an existing event with the same `seasonPlanEntryId` and updates it, or creates a new one. `clearSeasonPlanEntry()` deletes the linked event when a season plan entry is removed.

11. Teams Module

11.1 Team Configuration

`TeamPage.tsx` allows full configuration of a team. Each team stores:

Colors (per side)

- Primary (home jersey fill color)
- Secondary (unused visually, stored for future)
- Stroke ring color (border around player circle)
- Number text color
- All above for opponent side too

Roster & Schedule

- Player list with name, number, position
- Side assignment (home/opponent)
- Training days (e.g. Sunday, Tuesday, Thursday)

11.2 Color Propagation to Canvas

When a drill has `teamId` set to a team, the editor monitors the team's color keys via a `useEffect`. Any player with `teamColorInherited: true` gets automatically updated to match the team's current primary/stroke/number colors whenever those change. This means changing team colors in the Teams module reflects live in all linked drills.

12. Print System

12.1 Session Print

Route: `/sessions/[id]/print` → `SessionPrintView.tsx`

> The session print view generates a **branded A4 landscape document** with one cover page followed by one page per session block. All print layout is controlled via a `<style>` tag with `@media print` rules injected directly into the component.

Page	Content
Cover Page (`.cover-print-page`)	Session title, team name, date, total duration, block count, section breakdown table (warmup/main/game/cooldown with durations and intensity counts), brand header bar in #FF6A00
Each Drill Page (`.drill-print-page`)	Page header with session title, drill number/total; left column: drill name, duration, intensity badge, section phase badge, coaching points list, notes; right column: large MiniPitchPreview diagram (SVG), drill metadata (age group, player count, equipment)

```
/* Print CSS rules */
@media print {
  @page { size: A4 landscape; margin: 10mm 12mm; }
  .session-print-toolbar { display: none !important; }
  .drill-print-page {
    page-break-before: always;
    break-inside: avoid;
  }
  .cover-print-page { page-break-before: avoid; }
}
```

> The `.session-print-toolbar` class hides the Print / Back buttons when actually printing. The NavBar is hidden globally via a `.no-print` class applied in `globals.css`. This ensures a clean output with only the content pages.

12.2 Season Plan Print

Route: `/season-plans/[id]/view` → `SeasonPlanPrintView.tsx`

Renders a **week-by-week training table** designed for season overview printing. Weeks run Sunday-to-Saturday. Each week row spans the full page width. Cells show assigned sessions with drill thumbnail previews.

Element	Design
Document header	Team name, plan title, date range — background: `#263238`, text: white
Week header rows	Background: `#FF6A00`, white text, week number + date range
Training day cells	White background, session title, intensity badge, MiniPitchPreview SVG thumbnail
Empty day cells	Light gray (#F4F4F4) background, centered dash
Intensity badges	Low: blue (#dbeafe), Mid: amber (#fef3c7), High: red (#fee2e2)

12.3 MiniPitchPreview (SVG Thumbnail)

`MiniPitchPreview.tsx` renders a pure SVG representation of a drill's canvas objects — no Konva dependency. It is used in SessionTimeline block cards, DrillsList cards, and both print views.

Supported object types in SVG preview: players (circle + number text), cones (orange triangle), balls (white circle), goals (rectangle), arrows (SVG marker), lines, zones (filled rect), circles, rectangles, curved lines (SVG `<path>` with cubic bezier). Link objects are rendered as a line between player positions.

13. UI & Design System

13.1 Brand Color Palette

Primary	Dark UI	Background	Accent	Highlight	Canvas BG
#FF6A00	**#263238**	**#F4F4F4**	**#00B8D4**	**#FFC857**	**#1a1a2e**

Colors are registered as Tailwind custom tokens in `tailwind.config.ts`:

```
// tailwind.config.ts
theme: {
  extend: {
    colors: {
      'brand-orange': '#FF6A00',
      'brand-dark': '#263238',
    }
  }
}
```

13.2 Layout Structure

> The root layout (`app/layout.tsx`) renders a full-height flex row: **NavBar** (fixed left sidebar, 240px wide, `#263238` background) + **main content area** (flex-1, fills remaining width). The editor page adds a further horizontal split: PaletteSidebar (208px) + Canvas + InspectorPanel (288px).

Zone	Width	Background	Purpose
NavBar (left sidebar)	240px fixed	#263238	Global navigation: Drills, Sessions, Teams, Season Plans, Calendar
PaletteSidebar	208px fixed	gray-900	Tool palette for draw tools, equipment, shapes
Canvas area	flex-1 (remaining)	#1a1a2e (dark)	Konva Stage: pitch + objects
InspectorPanel	288px fixed	gray-900	Property inspector for selected objects

Zone	Width	Background	Purpose
Player Dock	full width, collapsible	gray-900	Player avatars + formation buttons
Session list pages	full width	#F4F4F4	White cards on light gray background

13.3 Component Patterns

Editor (Dark Theme)

The editor uses a dark UI: `bg-gray-900` panels, `bg-gray-800` inputs, emerald-500 active states, amber-500 draw-active states. All text uses white/gray variants.

List Pages (Light Theme)

Session Builder, DrillsList, SessionsList use a light theme: white cards (`bg-white`) on `#F4F4F4`, slate-200 borders, brand-orange accents, shadow-card utility class.

State Indicators

- Save status: "saving..." → "saved" auto-transition after 600ms debounce on any drill change
- Active draw tool: amber highlight in PaletteSidebar + amber status box below showing current tool name
- Active step tab: emerald highlight border on selected step
- Selected object: Konva Transformer renders handles around selected node

13.4 NavBar

The NavBar (`NavBar.tsx`) is a fixed left sidebar with:

- Brand logo mark ("C" in orange rounded square) + "CoachDesigner" wordmark
- 5 navigation links with emoji icons (⚽ Drills, 📋 Sessions, 👥 Teams, 🗓️ Season Plans, 🗓 Calendar)
- Active link: `bg-brand-orange/20 text-brand-orange`
- Inactive link: `text-white/60 hover:text-white hover:bg-white/10`
- Hidden on print via `.no-print` class

14. Current Capabilities

Drill Design

- Visual canvas editor (Konva)
- 4 pitch types: full, half, third, plain
- Custom pitch colors & grass stripes
- 11 canvas object types
- 7 tactical arrow/line types
- 3-click curved bezier line tool
- Smart arrow-to-player connections
- Player team assignment (A/B)
- GK detection (numbers 1, 12, 13)
- Global player scale slider (0.5×–2×)
- Zone areas with fill, opacity, label
- Shape tools: circle, rectangle
- Link objects between players

Editor Interaction

- 30-level undo/redo stack (per step)
- Multi-select (Shift+click)
- Marquee rubber-band selection
- Alignment snap (10px threshold)
- Shift+drag axis constraint
- Shift+draw 45° angle constraint
- Copy / paste with offset
- Duplicate (single + multi-select)
- Object lock/unlock
- Mouse wheel zoom
- Export PNG (2× pixel ratio)
- Alignment toolbar (8 modes)

Step & Progression System

- Base layer + unlimited progression steps
- Auto-clone base on step creation
- Clone base → current step action
- Step label editing
- Step-scoped undo/redo

Play Simulation

- rAF animation loop (1800ms)
- ease-in-out tween
- Players move along arrow paths
- Play / Stop controls

Drill Organisation

- Folder system with CRUD
- Subcategories per folder
- Favorites (star toggle)
- Drag-to-reorder (sortOrder)
- Drag between folders/subcategories
- 8 drill templates at creation
- Step count badge on card

Drill Metadata

- Objective, age group, player count
- Area size, duration
- Equipment list
- Coaching points, cues, mistakes
- Corrections, key constraints
- Progression/regression notes
- Tags, team link, training day
- Drill relationships (parent/variation)

Session Builder

- Session blocks with drill references
- 4 session sections (warmup/main/game/cooldown)
- Intensity per block (low/mid/high)
- Drag-to-reorder blocks
- Per-block notes
- Duplicate blocks
- Session duration summary
- Equipment aggregation

Teams, Season & Calendar

- Team color configuration (6 color fields)
- Player roster management
- Color inheritance to canvas players
- Season plan with date range
- Session assignment to training days
- Calendar auto-sync from season plan
- Calendar event status (planned/completed/cancelled)
- Month-view calendar

Print & Export

- Session print: cover + one page per drill
- Season plan print: week-by-week table
- A4 landscape with brand styling
- MiniPitchPreview in print output
- Drill PNG export (2× quality)
- Fullscreen drill view (Fullscreen API)

15. Limitations

1. Client-Only Storage (localStorage)

All data lives in the browser's localStorage. There is no backend, no database, and no cloud sync. Data is device-specific and will be lost if the browser data is cleared. There is no sharing, collaboration, or multi-device access.

2. No User Authentication

The application has no user accounts or login system. Anyone who opens the app on the same device has full access to all data. Multiple coaches cannot use separate workspaces on the same device.

3. localStorage Size Constraints

Browsers typically limit localStorage to 5–10MB per origin. Large drills with many objects or many drills/sessions stored simultaneously can approach this limit. There is no data compression or pagination of storage. The application will silently fail to save if the quota is exceeded (the storage adapter catches errors with empty catch blocks).

4. No Real-Time Collaboration

Multiple coaches cannot work on the same drill simultaneously. There is no conflict resolution, no operational transforms, and no WebSocket connection. This is a single-user offline tool.

5. Canvas Performance at Scale

The Konva canvas renders all objects in a single pass without virtualization. Drills with 50+ objects may experience performance degradation. Group move (multi-select drag) uses `stageRef.findOne()` for live updates which can be expensive on complex canvases.

6. SVG Print Limitations

The MiniPitchPreview used in print is SVG-based and does not perfectly match the Konva canvas rendering (especially for curved lines, rotated objects, and GK gold fill). Complex drills may look simplified in print.

7. No Drill Search/Filter

The drill library lacks a full-text search across all drills. Filtering is only by folder/subcategory selection. A coach with hundreds of drills cannot quickly find one by keyword, tag, or metadata.

8. No Versioning or History

There is no version history for drills beyond the in-session undo stack. Once the page is closed, the undo stack is lost. There is no "version history" for a drill over time — edits overwrite the stored state permanently.

9. Formation Coordinates Are Fixed

Formation templates use hardcoded percentage coordinates. There is no visual formation editor — coaches cannot create custom formations from scratch within the app.

10. No Mobile / Touch Optimisation

The canvas editor and multi-panel layout are designed for desktop use. The application is not optimised for touch input, mobile screen sizes, or stylus/tablet drawing. Konva touch events are not configured.

11. Session Quality Analysis is Minimal

`src/lib/sessionQuality.ts` exists but the analysis logic is basic. There are no AI-driven insights, no intensity curve visualisation, and no warm-up/cool-down time recommendations based on session load.

16. Professional Improvement Suggestions

1

Cloud Backend & Multi-Device Sync

Replace localStorage with a proper backend (Supabase, PlanetScale, or a Node.js/PostgreSQL API). Use Zustand middleware to sync store mutations to the server via REST or GraphQL. This enables multi-device access, backup, and future collaboration. A lightweight approach: Supabase with Row Level Security + Zustand's `subscribeWithSelector` for optimistic UI.

2

User Authentication & Multi-Coach Workspaces

Add authentication (NextAuth.js or Supabase Auth) to enable multiple coaches per club to have isolated workspaces. Add coach profiles, admin roles, and club-level sharing. This is the single highest-value change to make the product multi-tenant and commercially viable.

3

Real-Time Collaborative Editing

Implement operational transforms or CRDTs (e.g. Yjs) over WebSockets (Socket.io or Partykit) to allow multiple coaches to edit the same drill canvas simultaneously. Show live cursors and user avatars on the canvas. This elevates the platform to a professional coaching tool comparable to Miro or Figma for sports.

4

AI Drill Generation & Coaching Assistant

Integrate the Claude API or OpenAI API to provide: (a) natural language drill generation ("Create a 4v2 rondo with pressing triggers"), (b) automatic coaching point suggestions based on the drill type, (c) session structure recommendations based on player age/objective. This transforms the tool from a drawing app into an intelligent coaching assistant.

5

Canvas Performance & Infinite Canvas

Implement canvas virtualisation for large drills: only render objects within the current viewport. Add infinite pan with minimap navigation. Use Konva's built-in caching (`node.cache()`) for static pitch layer. Consider a WebGL renderer (Pixi.js) for very large formation diagrams or animation-heavy simulations.

6

Video & Animation Export

Extend the Play Simulation system to record frames and export as a GIF or MP4 video. Use `MediaRecorder API` with canvas `captureStream()`. Coaches could share animated drill previews directly to messaging platforms. Add multi-step sequential animation (play step 1, then step 2 automatically).

7

Full-Text Drill Search & Smart Filtering

Add a search index (Fuse.js for client-side fuzzy search) across all drill fields: title, description, objective, tags, coaching points. Add filter chips for age group, player count, duration range, intensity, equipment. A coach with 200+ drills needs to find "4v2 rondo with GK" in under 2 seconds.

8

Custom Formation Editor

Replace the hardcoded formation templates with a visual formation builder: drag players freely on a mini-pitch to define custom formations, then save them as named templates. Support tactical variations within a formation (e.g. defensive shape vs. attacking shape from the same formation base).

9

Drill Versioning & History

Store a version history for each drill (e.g. last 20 saves with timestamps). Allow coaches to browse history, preview, and restore a previous version. This is critical for professional use where coaches frequently iterate drills over a season and may want to revert to an earlier concept.

10

Training Load Analytics Dashboard

Add an analytics view showing: weekly/monthly intensity distribution across sessions, training volume trends, intensity periodization chart (tapering into matchdays), most-used drills and equipment. Integrate with the season plan and calendar to derive load data automatically from the stored sessions.

11

Mobile & Tablet Optimisation

Redesign the UI with responsive breakpoints and touch-first interactions for iPad/tablet use. Konva supports touch events — add pinch-to-zoom, two-finger pan, and tap-to-select. A touchscreen canvas editor would allow coaches to design drills directly from the pitch with a tablet. This opens the product to a significantly larger user base.

12

Drill Library Sharing & Community

Build a public drill library where coaches can publish drills with a one-click "share". Add import/export (JSON or custom format) for individual drills or entire season plans. Create a community browser where coaches can discover and fork drills from others. This creates network effects and makes the platform defensible.

13

GPS / Wearable Data Integration

For advanced clubs, integrate with GPS tracking providers (STATSports, Catapult, Polar) to import actual training load data and overlay it against planned sessions. Show planned-vs-actual intensity, distance, and HRV recovery data alongside the season plan.